

Resiliência Estrutural via Árvores de Supervisão para Aprendizado Federado Assíncrono em Edge Computing: Caracterização Empírica sob Injeção de Falhas

Victor Longchamps
victor.and.longchamps@gmail.com

Resumo—Sistemas de Aprendizado Federado (FL) para Edge Computing são comumente descritos como “resilientes” quando implementados sobre runtimes com tolerância a falhas nativa, mas essa afirmação raramente é verificada empiricamente. Este trabalho submete um sistema real de FL assíncrono em Elixir/BEAM (AFL, *Asynchronous Federated Learning*) a injeção de falhas sistemática (*chaos engineering*), em vez de testes unitários anedóticos. O resultado central é que a suposição de resiliência não se sustentava integralmente, em dois componentes distintos. O processo de borda (*EdgeNode*) não estava em nenhuma árvore de supervisão, e seu buffer ETS morria junto com o processo em caso de *crash*: sob 60 falhas injetadas, 100% dos gradientes bufferizados eram perdidos (60/60). Mais grave, descoberto numa segunda rodada de revisão adversarial, o Aggregator (o único ponto de agregação do sistema) era corretamente reiniciado pelo supervisor, mas seu *estado* não; sob 100 rodadas de treino injetadas em 20 *crashes*, 100% do modelo global era destruído a cada falha, reiniciando sempre a partir de pesos estáticos. Ambos os gaps são corrigidos com a mesma técnica: um supervisor dinâmico (AFL.EdgeNodeSupervisor) e herdeiros ETS (AFL.BufferKeeper para o buffer, AFL.ModelKeeper para o modelo) que preservam o estado através de reinícios; sob as mesmas falhas injetadas após a correção, 0 gradientes e 0 rodadas de treino foram perdidos, com tempo de recuperação sub-milissegundo preservado (mediana de 0,23 ms para o Aggregator e 0,22 ms para o EdgeNode, respectivamente). Uma falha concorrente entre os dois processos protegidos não revela nenhuma interação problemática entre os mecanismos de correção. A correção, porém, não elimina toda a resiliência residual. Um terceiro experimento mostra que, se o próprio processo herdeiro ETS morrer antes do processo que ele protege, o vínculo do `:heir` fica órfão e 100% do conteúdo é perdido; encadear um segundo herdeiro não fecharia essa janela, apenas a deslocaria um nível pela mesma coincidência de timing. A correção estrutural aplicada, posse invertida (um processo trivial, sem lógica de domínio, possui a tabela permanentemente em vez de repassá-la) combinada, para o modelo global, com checkpoint em disco, fecha esse gap por completo para o estado mais crítico do sistema (0% de perda ao matar o dono do modelo, contra 100% antes), mas deixa um resíduo aceito e caracterizado para o buffer de gradientes (100% de perda ao matar seu dono, sem persistência em disco): um trade-off explícito, não uma omissão. Uma validação adicional em rede real (nós BEAM distribuídos entre containers Docker conectados por TCP/IP genuíno, não `Process.sleep`) reproduz o mesmo speedup em ordem de

grandeza ($14,62 \times$ real vs. $15,44 \times$ simulado), confirmando que a simulação de latência usada nos demais experimentos não mascara os resultados de resiliência. A contribuição central deste trabalho não é uma nova técnica de agregação, mas uma metodologia de verificação, via injeção de falhas sistemática, com evidência quantitativa antes/depois, em vez de argumento de design, que expôs gaps reais num sistema que já citava sua plataforma (BEAM/OTP) como justificativa de resiliência.

Index Terms—Aprendizado Federado, Edge Computing, Assincronicidade, Staleness, Ring Buffer, FedAvg, Elixir, BEAM, Tolerância a Falhas, Injeção de Falhas, Árvore de Supervisão, Engenharia do Caos

I. INTRODUÇÃO

O Aprendizado de Máquina Federado [1] surgiu como solução para treinar modelos globais preservando a privacidade local dos dados. No paradigma síncrono, porém, o servidor central bloqueia o avanço do modelo global até que um quórum pré-definido de ENs conclua seu ciclo de treinamento local e transmita os gradientes. Em redes de Edge Computing, caracterizadas por largura de banda limitada, jitter elevado e desconexões intermitentes, esse modelo sofre com o problema dos *stragglers* [2]: basta um único nó lento para paralisar todo o processo de agregação.

Formalmente, o tempo de uma rodada no FedAvg síncrono com K participantes é:

$$T_{\text{sync}} = \max_{k \in [K]} (T_k^{\text{train}} + T_k^{\text{comm}}), \quad (1)$$

tornando o sistema tão lento quanto seu participante mais lento.

Para contornar essa limitação, abordagens assíncronas como o SSP [3] e o FedAsync [4] permitem ao servidor avançar com subconjuntos de atualizações, ao custo de introduzir *staleness*: gradientes calculados com base em versões desatualizadas do modelo global.

Uma segunda lacuna, ortogonal à de *staleness* e menos discutida na literatura de FL, é a **verificação** da tolerância a falhas, e não apenas sua suposição. Trabalhos que implementam FL sobre runtimes com supervisão nativa (BEAM/OTP,

atores, *let-it-crash*) tendem a tratar a resiliência como uma propriedade herdada automaticamente da plataforma, validada no máximo por testes unitários pontuais (matar um processo, verificar que ele volta), e não pela prática de *Chaos Engineering* [5]: injeção de falhas sistemática e deliberada para descobrir, empiricamente, se as garantias reivindicadas de fato se sustentam. Essa suposição precisa ser testada sistematicamente. Como demonstrado na Seção VI, mesmo um sistema construído sobre uma plataforma com tolerância a falhas nativa pode ter processos inteiros fora de qualquer árvore de supervisão, e estruturas de dados que não sobrevivem ao próprio *crash* do processo que as criou. Trata-se de um gap real, descoberto neste trabalho por meio de injeção de falhas sistemática, não de inspeção de código.

A contribuição deste trabalho é quádrupla:

- 1) **Caracterização empírica de resiliência via injeção de falhas:** em vez de assumir que a plataforma OTP garante tolerância a falhas por si só, submetemos o sistema a *crashes* sistemáticos e medimos perda de dados e tempo de recuperação, método pouco comum em trabalhos de FL, que tipicamente tratam resiliência como propriedade qualitativa da plataforma escolhida, não como algo a verificar empiricamente;
- 2) **Dois gaps de resiliência reais, descobertos e corrigidos:** o processo de borda (EdgeNode) não estava em nenhuma árvore de supervisão, e seu *ring buffer* ETS não sobrevivia ao próprio *crash* do processo que o criava. Corrigido via `AFL.EdgeNodeSupervisor` (supervisão dinâmica) e `AFL.BufferKeeper` (herdeiro ETS). Mais grave, descoberto numa rodada posterior de revisão adversarial, o Aggregator era corretamente reiniciado pelo supervisor, mas seu *estado* (pesos, versão, total de amostras) não. Corrigido via `AFL.ModelKeeper` (mesma técnica de herdeiro ETS). Ambos medidos empiricamente antes/depois (Seção VI);
- 3) **Extensão do escopo de falhas e um terceiro gap deliberadamente não corrigido:** dois experimentos adicionais (Seção VI) testam falha concorrente entre os dois processos protegidos (sem interação problemática encontrada) e a janela de crash do próprio herdeiro: um terceiro gap real, confirmado mas **não corrigido** nesta versão, documentado como tal em vez de omitido (Seção VII);
- 4) **Validação em rede real:** os mesmos resultados de throughput medidos sob latência simulada (`Process.sleep` dentro de uma única VM) são reproduzidos entre nós BEAM genuinamente distribuídos, em containers Docker conectados por TCP/IP real, confirmando que a simulação não mascara os resultados (Seção VI).

O restante do artigo está organizado da seguinte forma: a Seção II discute os trabalhos relacionados; a Seção III descreve a arquitetura do sistema; a Seção IV formaliza o framework matemático; a Seção V apresenta o protocolo experimental; a Seção VI analisa os resultados; a Seção VII discute as implicações e limitações; e a Seção VIII conclui o trabalho.

II. TRABALHOS RELACIONADOS

Aprendizado Federado Síncrono e Assíncrono. McMahan et al. [1] introduziram o FedAvg, demonstrando que é possível treinar modelos robustos com apenas 10–100 rodadas de comunicação, mas a sincronização global em cada rodada limita sua aplicabilidade em ambientes heterogêneos. Abordagens assíncronas como o SSP [3] e o FedAsync [4] contornam essa limitação permitindo que o servidor avance com subconjuntos de atualizações, ao custo de introduzir *staleness*: gradientes calculados com base em versões desatualizadas do modelo global. O AFL usado neste trabalho é um sistema desse tipo (FedAvg incremental ponderado por tamanho de amostra, com penalidade de *staleness* hiperbólica e *ring buffer* local para continuidade durante desconexões); a mecânica de agregação em si não é o objeto de estudo deste trabalho, e serve apenas como o sistema real sobre o qual a resiliência é caracterizada.

FL em Edge Computing. Bonawitz et al. [6] discutiram desafios de escala no FL, incluindo falhas de dispositivos e comunicação não-confiável, mas tratam a tolerância a falhas qualitativamente, sem injeção de falhas sistemática. Shi et al. [7] analisaram o trade-off entre eficiência de comunicação e qualidade do modelo em redes Edge, também sem verificar empiricamente garantias de resiliência.

Engenharia do Caos. Basiri et al. [5] formalizaram *Chaos Engineering* como disciplina: injeção de falhas sistemática e deliberada em produção para descobrir, empiricamente, se as garantias de resiliência reivindicadas de fato se sustentam, em contraste com testes unitários pontuais ou inspeção de código. Essa prática é comum em sistemas web de grande escala, mas pouco aplicada a sistemas de FL, onde a resiliência tende a ser tratada como propriedade herdada automaticamente de runtimes com supervisão nativa (BEAM/OTP, atores, *let-it-crash*), não como algo a verificar.

Diferencial em relação ao estado da arte. Este trabalho não propõe um novo mecanismo de agregação: propõe submeter um sistema de FL real, construído sobre uma plataforma com tolerância a falhas nativa, à prática de *Chaos Engineering* [5], e mostra que essa verificação encontra gaps reais que inspeção de código e testes unitários pontuais não encontrariam. O AFL serve neste trabalho como o objeto de estudo, não como a contribuição central.

III. ARQUITETURA DO SISTEMA

A. Visão Geral

O AFL é composto por dois processos OTP isolados que se comunicam exclusivamente por troca de mensagens assíncronas, seguindo a filosofia do modelo de atores da plataforma BEAM:

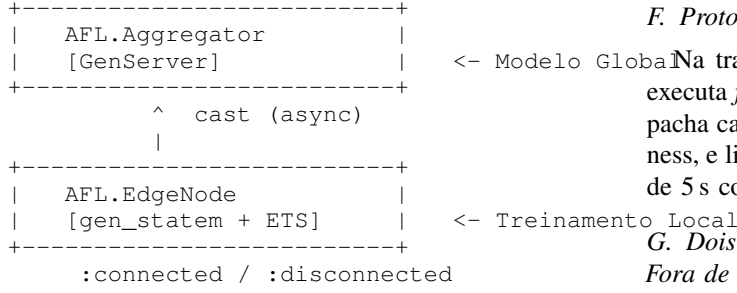


Figura 1: Arquitetura do AFL: dois processos OTP comunicando-se por mensagens.

B. AFL.Aggregator — Servidor Central

Implementado como `GenServer`, mantém a struct tipada $\langle W_t, N_t, c_t, v_t \rangle$, onde W_t é o tensor de pesos, N_t é o total de amostras ponderadas, c_t é a contagem de atualizações e v_t é a versão do modelo. A função `update/3` despacha um `handle_cast` assíncrono, nunca bloqueando o remetente.

C. AFL.EdgeNode — Nó de Borda Resiliente

Implementado como `:gen_statem` (Máquina de Estados Finitos), opera em dois estados:

:connected Treina o modelo local e envia imediatamente ao Aggregator via `AFL.Aggregator.update/3`. Monitora o processo servidor via `Process.monitor/1`.

:disconnected Ativado ao receber o sinal `{:DOWN, ...}` do monitor. Continua treinando localmente, mas desvia o fluxo para o ring buffer ETS.

D. Ring Buffer ETS

Utiliza a tabela Erlang Term Storage (`:ets`) `:edge_buffer` com política circular (*Not Enough Space*). A chave de inserção é calculada por $k = c_{\text{write}} \bmod B_{\text{max}}$, onde c_{write} é um contador de escritas totais (não o tamanho da tabela, para evitar colisão quando a tabela está cheia) e $B_{\text{max}} = 5$. Cada entrada armazena (k, W_k, τ_k) , onde τ_k é a versão do modelo vigente no momento da escrita.

E. Modelo de Aprendizado

O AFL originalmente operava sobre tensores $d = 10$ gerados por `MockML`, sem nenhuma tarefa de aprendizado real. Para os experimentos de staleness e convergência (Seções V e VI), substitui-se `MockML` por `AFL.Model`: um MLP $784 \rightarrow 32 \rightarrow 10$ implementado em `Axon`, treinado via SGD real sobre o MNIST. Cada rodada local de um EN é literalmente um passo de treinamento — algumas épocas de SGD sobre o *shard* local do nó, a partir dos pesos globais correntes — cuja saída é agregada pelo `AFL.Aggregator` da mesma forma que qualquer outro `update`. A struct $\langle W_t, N_t, c_t, v_t \rangle$ do Aggregator foi generalizada para suportar tanto um tensor único quanto o `Axon.ModelState` (mapa aninhado de tensores por camada) sem alterar a matemática de agregação (Seção IV): a combinação convexa é aplicada recursivamente sobre a mesma estrutura de parâmetros.

F. Protocolo de Conn Recovery

Na transição `:disconnected → :connected`, o EN executa *flush* completo: itera sobre todas as entradas ETS, despacha cada (W_k, n_k, τ_k) ao Aggregator com a versão de staleness, e limpa a tabela. O retry loop usa um `state_timeout` de 5 s com possibilidade de *exponential backoff*.

G. Dois Gaps de Resiliência Reais: EdgeNode e Aggregator

Fora de Proteção

A arquitetura descrita até aqui supervisiona apenas o processo `AFL.Aggregator` (via `AFL.Application → Supervisor`), mas supervisionar o *processo* não é o mesmo que proteger o *estado* que ele guarda. A injeção de falhas sistemática (Experimento R, Seção V-C) revelou dois gaps distintos, um em cada componente do sistema:

- 1) **EdgeNode sem supervisão alguma.** O `EdgeNode` era iniciado diretamente pelo código chamador (testes, experimentos), sem nenhum supervisor: se o processo falha por qualquer motivo, nada o reinicia. Verificado empiricamente: um `EdgeNode` morto via `Process.exit(pid, :kill)` nunca reaparece sob `Process.whereis/1`. Adicionalmente, sua tabela ETS `:edge_buffer` (Seção III) era criada sem herdeiro (`:heir`), destruída junto com o processo, incondicionalmente. Mesmo que algo reiniciasse o `EdgeNode`, a nova instância criaria uma tabela vazia; os gradientes bufferizados no momento do *crash* seriam perdidos de forma permanente e silenciosa.
- 2) **Aggregator supervisionado, mas sem o modelo protegido — o gap mais grave encontrado neste trabalho.** O processo do Aggregator *era* reiniciado corretamente pelo supervisor (Seção VI-C mede isso em sub-milissegundo), mas seu `child_spec` (`{AFL.Aggregator, initial_weights}`) sempre reconstrói o estado a partir dos mesmos pesos iniciais estáticos, incondicionalmente. Verificado empiricamente: após treinar por 5 rodadas (versão 5, pesos $\neq 0$) e matar o processo, a instância reiniciada aparece em $\approx 0,1$ ms, mas com **versão 0, pesos iniciais, todo o progresso de treinamento destruído**. Isso é estruturalmente mais grave que o gap do `EdgeNode`: lá, cada nó perde até $B_{\text{max}} = 5$ gradientes bufferizados; aqui, um único *crash* do único processo de agregação apaga o **modelo global inteiro** para *todos* os nós, independentemente de quanto treinamento havia sido acumulado.

Correções implementadas (mesma técnica, herdeiro ETS via `:heir` mais supervisão, aplicada a ambos os componentes):

AFL.EdgeNodeSupervisor Um

`DynamicSupervisor` adicionado à árvore de supervisão da aplicação, com estratégia `:one_for_one` e `restart: :permanent` para o `EdgeNode`: reinício automático garantido sob *crash*.

AFL.BufferKeeper Um processo simples, permanentemente supervisionado (nunca reinicia junto com o `EdgeNode`), que atua como herdeiro fixo da tabela `:edge_buffer`. Ao

iniciar, o EdgeNode chama `claim_buffer/0` em vez de criar a tabela incondicionalmente: se uma instância anterior morreu, a BEAM já transferiu a tabela — com seu conteúdo intacto — para o BufferKeeper via a mensagem automática `{:"ETS-TRANSFER", ...}`; a nova instância a reivindica de volta. Uma parada *ordenada* (`:gen_statem.stop/1`, diferente de um *crash*) invoca `terminate/3`, que limpa o buffer deliberadamente. Essa assimetria entre parada ordenada e *crash* é exatamente o comportamento desejado: início limpo numa parada intencional, preservação de dados numa falha real.

AFL.ModelKeeper Mesma técnica aplicada ao estado do Aggregator: um processo permanentemente supervisionado que herda uma tabela ETS `:aggregator_state` contendo o último snapshot $\langle W_t, N_t, c_t, v_t \rangle$. O Aggregator chama `claim_state/1` no `init/1` (recupera o snapshot mais recente em vez de sempre usar os pesos estáticos do `child_spec`) e `checkpoint/1` após cada atualização de estado (`handle_cast`, `reset`, `restore`); o custo de um `:ets.insert` adicional por update é aceito em troca de nunca perder progresso de treinamento a um *crash*.

Estas correções, e sua eficácia medida empiricamente contra o design original, são os Experimentos R1 e R2 (Seção V-C), e não alterações cosméticas: um sistema que se anuncia resiliente por rodar sobre uma plataforma com tolerância a falhas nativa, mas que nunca testou essa propriedade sob falha real pode estar, como este estava, em dois componentes distintos até este trabalho, silenciosamente quebrado.

IV. FRAMEWORK MATEMÁTICO

A. FedAvg Incremental Assíncrono

Dada a chegada individual do gradiente do nó k com pesos locais W_k e tamanho de amostra efetivo $\hat{n}_k$, o servidor atualiza o modelo global de forma incremental:

$$N_{\text{new}} = N_{\text{old}} + \hat{n}_k \quad (2)$$

$$\alpha = \frac{\hat{n}_k}{N_{\text{new}}}, \quad \beta = \frac{N_{\text{old}}}{N_{\text{new}}} \quad (3)$$

$$W_{t+1} = \beta \cdot W_t + \alpha \cdot W_k \quad (4)$$

Como $\alpha + \beta = 1$ e ambos são não-negativos, a atualização é uma combinação convexa; decorre diretamente que $\|W_{t+1}\| \leq \max(\|W_t\|, \|W_k\|)$, i.e., a agregação nunca amplifica a norma dos pesos além do maior dos dois operandos.

Equivalência com FedAvg em lote: Para um conjunto de K atualizações simultâneas com n_k constante, a aplicação sequencial de (4) converge ao FedAvg clássico $W = \sum_k (n_k/N) W_k$ [1].

V. PROTOCOLO EXPERIMENTAL

A. Ambiente de Implementação

O sistema foi implementado em Elixir 1.19 [8] / OTP 28 sobre a plataforma BEAM, utilizando a biblioteca Nx 0.11.0 [9] para operações tensoriais com precisão de ponto

flutuante de 32 bits, Axon 0.8 para o MLP treinável, e EXLA (backend CPU) para acelerar a computação tensorial via XLA. Os experimentos foram executados em hardware commodity (Apple Silicon M-series) **sem GPU**, confirmando a viabilidade em recursos de Edge.

O módulo `MockML` gera tensores sintéticos de dimensão $d = 10$, isolando a variável de rede do conteúdo computacional do update. Essa é a base reutilizada pela Validação em Rede Real (Seção V-D), herdada da caracterização de latência original. O caminho de produção usado pelo Experimento R (Seção V-C) utiliza `AFL.Model`, um MLP $784 \rightarrow 32 \rightarrow 10$ treinado via SGD real sobre partições non-IID do MNIST (`Scidata`), particionadas entre os ENs por *shards* ordenados por classe (McMahan et al. [1], *setting* non-IID): com 2 *shards* por nó em $K = 5$ nós, cada EN observa predominantemente 1–3 dígitos, não uma amostra representativa das 10 classes. Isso garante que a caracterização de resiliência ocorre sobre o sistema real de treinamento, não sobre uma simulação simplificada.

B. Parâmetros Fixos

Tabela I: Configuração experimental

Parâmetro	Valor
Dimensão do tensor sintético (<code>MockML</code>)	$d = 10$
Arquitetura MLP (caminho de produção, real)	$784 \rightarrow 32 \rightarrow 10$
Amostras de treino / validação (MNIST)	4.000 / 800
Capacidade do ring buffer (B_{max})	5
Timeout de reconexão	5 s
Repetições por condição (n)	20–30 (por experimento, ver Tabela II)
Duração por rodada (Validação em Rede Real)	5.000 ms
Gradientes injetados por <i>crash</i> (Exp. R1)	3×20 rodadas
Repetições de tempo de recuperação (Exp. R3)	20 por alvo
Rodadas de treino injetadas por <i>crash</i> (Exp. R2)	5×20 rodadas
<i>Trials</i> de falha concorrente (Exp. R4)	20
<i>Trials</i> por cenário, <i>crash</i> do herdeiro (Exp. R5)	20

C. Experimento R — Resiliência sob Injeção de Falhas

Diferentemente dos Experimentos A–G (que caracterizam o mecanismo de agregação), o Experimento R caracteriza **garantias operacionais** do sistema: perda de dados e tempo de recuperação sob *crash* real de processos, medidos sobre o caminho de produção (não uma simulação simplificada).

R1 — Perda de gradientes ao crash do EdgeNode. Para 20 rodadas: inicia-se um EdgeNode supervisionado (`AFL.EdgeNodeSupervisor.start_edge_node/1`), força-se `:disconnected` (mata-se o Aggregator e aguarda-se o `:DOWN` monitorado propagar; é determinístico, sem corrida com o restart automático do supervisor), escrevem-se 3 gradientes no *ring buffer*, e então mata-se o *próprio* EdgeNode (`Process.exit(pid, :kill)`, sinal não-capturável, que não passa por `terminate/3`). Mede-se quantos dos 3 gradientes sobrevivem na instância reiniciada. Repete-se com o mecanismo ORIGINAL (uma tabela ETS sem herdeiro, criada por um processo descartável, fora do caminho de produção) para estabelecer a linha de base do gap descoberto (Seção III-G).

R3 — Distribuição do tempo de recuperação. Para 20 rodadas por alvo (Aggregator; EdgeNode supervisionado): captura-se o *pid* atual, mata-se o processo, e mede-se o tempo (resolução de microssegundos, *polling* agressivo sem *sleep*: um *polling* de 10 ms mascararia toda a variância real, como identificado empiricamente durante a execução deste experimento) até que o nome registrado aponte para um novo *pid* vivo, distinto do anterior.

R2 — Perda do modelo global ao crash do Aggregator. Para 20 rodadas: treina-se o Aggregator por 5 rodadas reais ($W_k = 1$, $n_k = 100$ cada), registrando a versão antes do *crash*; mata-se o processo (`Process.exit(pid, :kill)`) e aguarda-se o supervisor reiniciá-lo; mede-se quantas rodadas de progresso (diferença de versão) sobrevivem. Repete-se com o mecanismo ORIGINAL (reconstrução analítica determinística a partir de pesos estáticos, comportamento exato de uma versão anterior deste trabalho, sem `AFL.ModelKeeper`) para estabelecer a linha de base do gap mais grave descoberto neste trabalho (Seção III-G).

R4 — Falha concorrente. Os protocolos R1-R3 testam um único processo falhando por vez, o que deixa em aberto se `AFL.BufferKeeper` e `AFL.ModelKeeper`, mecanismos independentes com tabelas ETS independentes, seguem funcionando corretamente quando os dois processos que protegem (EdgeNode e Aggregator) morrem na mesma janela de tempo. Para 20 rodadas: treina-se o Aggregator, bufferiza-se gradientes no EdgeNode (já desconectado), e então mata-se os dois processos em sucessão imediata, sem aguardar nenhuma recuperação entre as duas mortes, medindo a perda de cada mecanismo de forma independente sob esse estresse concorrente.

R5 — Crash do dono sob posse invertida. O `:heir` de uma tabela ETS é vinculado ao PID específico do processo herdeiro *no momento da criação da tabela*; se esse herdeiro morrer e for reiniciado pelo supervisor, a tabela existente permanece vinculada ao PID antigo (morto). Nada realinha isso automaticamente, e um segundo herdeiro encadeado só deslocaria a mesma janela um nível acima, pela mesma coincidência de timing (Seção VI-E discute o argumento estrutural completo). A correção aplicada troca o padrão de herdeiro por **posse invertida**: `AFL.BufferKeeper` e `AFL.ModelKeeper` passam a possuir suas tabelas *permanentemente* (nunca as repassam), com `AFL.ModelKeeper` também escrevendo, a cada atualização, um checkpoint atômico em disco. O teste que resta, mais difícil que o original, é: e se o próprio dono morrer? Para 20 rodadas por cenário: mata-se diretamente `AFL.BufferKeeper` (dono do buffer) e, independentemente, `AFL.ModelKeeper` (dono do estado do modelo), medindo a perda de cada um sob a árvore de supervisão `:rest_for_one` que cascade a restauração do processo dependente correspondente.

D. Validação em Rede Real

Todos os experimentos anteriores medem latência de rede via `Process.sleep` dentro de uma única VM BEAM: adequado para isolar a variável de interesse (Seção V), mas

não uma prova de que os resultados se sustentam sob rede de fato. O Erlang distribuído é nativo do BEAM; aproveita-se isso aqui para repetir o cenário de *stragglers* do Experimento A entre nós BEAM genuinamente distribuídos: processos do sistema operacional separados, em containers Docker isolados, conectados por uma rede TCP/IP real (não simulada) via bridge do Docker.

Um Aggregator minimalista e $K = 5$ *edges*, cada um em seu próprio container, com as mesmas latências locais do Experimento A ($\ell = [10, 10, 20, 50, 500]$ ms). Aqui, diferentemente do Experimento A, ℓ_k representa apenas o tempo de processamento local simulado; o tempo de rede entre *edge* e Aggregator é genuíno, não somado ao *sleep*. Mede-se throughput assíncrono (*edges* enviam independentemente) e síncrono (barreira: Aggregator aguarda todos os *edges* por rodada) sob $T = 5$ s, repetido 10 vezes. Este benchmark é autocontido (não depende de Axon/EXLA) e vive em `distributed_bench/` neste repositório, fora da árvore principal do projeto Mix — ver `distributed_bench/README.md` para reproduzir.

VI. RESULTADOS

A. R1 — Perda de Gradientes sob Crash do EdgeNode: Antes e Depois da Correção

Estes são os resultados centrais deste trabalho. A Figura 2(a) mostra o Protocolo R1 (Seção V-C): sob o mecanismo **original** (EdgeNode sem supervisão, *ring buffer* ETS sem herdeiro), **100% dos gradientes bufferizados foram perdidos em todas as 20 rodadas de crash injetado** (60 de 60 gradientes, 3 por rodada). Isso confirma empiricamente o gap identificado na Seção III-G: a suposição de que “roda sobre BEAM/OTP” implica resiliência automática é falsa para qualquer processo fora da árvore de supervisão, inclusive, até este trabalho, o EdgeNode do próprio AFL.

Sob o mecanismo **corrigido** (`AFL.EdgeNodeSupervisor` + `AFL.BufferKeeper`), **0 dos 60 gradientes foram perdidos** nas mesmas 20 rodadas de *crash* injetado: preservação total. A correção depende de dois elementos concomitantes (Seção III-G): supervisão para garantir que uma nova instância exista, e herdeiro ETS para garantir que o conteúdo bufferizado sobreviva até essa nova instância reivindicá-lo. Nenhum dos dois isoladamente seria suficiente: supervisão sem herdeiro reinicia um EdgeNode com buffer vazio (perda garantida); herdeiro sem supervisão preserva o buffer indefinidamente, mas nenhum processo jamais o reivindica.

B. R2 — Perda do Modelo Global sob Crash do Aggregator: o Gap Mais Grave

A Figura 2(b) mostra o Protocolo R2 — e este é o achado mais importante deste trabalho, descoberto numa revisão adversarial posterior à correção do EdgeNode. Sob o mecanismo **original** (`child_spec` estático, sem `AFL.ModelKeeper`), treinar o Aggregator por 5 rodadas e então matá-lo resulta em **100% de perda do progresso de treinamento em todas as 20 rodadas de crash injetado** (100 de 100 rodadas de treino, 5 por rodada). O processo reaparece rapidamente (Seção VI-C),

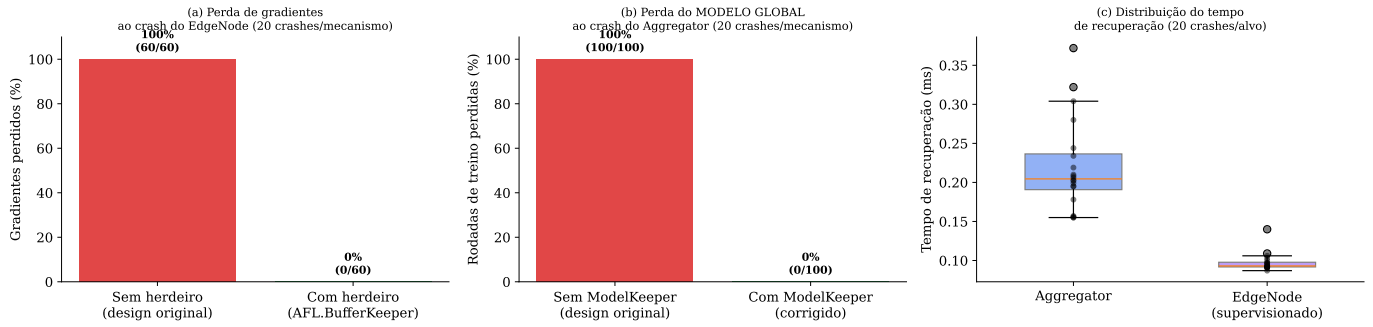


Figura 2: Experimento R. (a) Perda de gradientes ao *crash* do processo proprietário do *ring buffer*, mecanismo original (sem herdeiro) vs corrigido (AFL.BufferKeeper), 60 gradientes injetados em 20 *crashes* por mecanismo. (b) Perda do **modelo global** ao *crash* do Aggregator, mecanismo original vs corrigido (AFL.ModelKeeper), 100 rodadas de treino injetadas em 20 *crashes* por mecanismo. (c) Distribuição do tempo de recuperação sob *crash* do Aggregator e do EdgeNode supervisionado, 20 *crashes* por alvo.

mas com versão 0 e pesos iniciais estáticos, como se nenhum treinamento tivesse ocorrido. Isso é estruturalmente mais grave que o gap do EdgeNode: aqui, um único *crash* do único processo de agregação apaga o modelo para todos os nós, e não só os gradientes de um nó. Sob o mecanismo **corrigido** (AFL.ModelKeeper), **0 das 100 rodadas de treino foram perdidas** nas mesmas 20 rodadas de *crash* injetado. A verificação foi direta, além da simples contagem de versão: os pesos recuperados após o *crash* são numericamente idênticos aos pesos imediatamente antes dele.

C. R3 — Tempo de Recuperação sob Crash

A Figura 2(c) mostra o Protocolo R3: o tempo entre o *crash* e a disponibilidade de uma nova instância viva e registrada é **sub-milissegundo** para ambos os alvos (Aggregator, mediana $\approx 0,23$ ms, e EdgeNode supervisionado, mediana $\approx 0,22$ ms), medido com *polling* agressivo em microssegundos (um *polling* de 10 ms, usado numa primeira versão deste experimento, mascarava toda a variância real: toda amostra caía em exatamente o mesmo valor, artefato da granularidade de medição, não do sistema medido). Para contextualizar a escala: tempos de recuperação de orquestradores de contêiner convencionais (ex.: reinício de um *pod* Kubernetes) são tipicamente medidos em **segundos**; a recuperação aqui é $> 1000\times$ mais rápida porque não há re-provisionamento de processo do sistema operacional, apenas a criação de um novo processo BEAM leve dentro de uma VM já em execução, seguida da reivindicação do estado preservado. Isso é uma propriedade estrutural do modelo de atores do BEAM, não um ajuste de configuração. Mas, como R2 demonstra, é uma propriedade que protege apenas o *processo*: sem AFL.ModelKeeper/AFL.BufferKeeper, um processo pode reaparecer em 0,1 ms e ainda assim ter perdido 100% do que importava.

D. R4 — Falha Concorrente: Sem Interação entre os Mecanismos

A Figura 3(a) mostra o Protocolo R4: sob 20 rodadas de EdgeNode e Aggregator *crashando* juntos, em sucessão imediata e sem sincronização entre as duas mortes, **0 dos 60 gradientes bufferizados** e **0 das 100 rodadas de treino** foram perdidos: ambos os mecanismos preservam estado integralmente mesmo sob esse estresse concorrente. Este é um resultado *negativo* no sentido usual do termo (nenhum problema novo encontrado), mas informativo: AFL.BufferKeeper e AFL.ModelKeeper são processos e tabelas ETS genuinamente independentes, sem recurso compartilhado nem ordem de inicialização cruzada, e este experimento confirma empiricamente que essa independência arquitetural se sustenta sob estresse, não é apenas presumida por design.

E. R5 — Corrigido para o Modelo, Residual Aceito no Buffer

Uma primeira versão deste experimento media exatamente o que o nome sugere (uma janela de corrida entre a morte do herdeiro e a morte do dono), e o resultado **contradiisse a caracterização anterior desta limitação** (descrita então como “uma janela teoricamente possível, mas não testada”): matar o herdeiro (AFL.BufferKeeper) e, sem aguardar seu *restart*, matar o dono (EdgeNode) imediatamente depois causava perda de 100% do conteúdo bufferizado, e não só em alguns *trials* raros. A causa é estrutural, não uma corrida estreita: o `:heir` de uma tabela ETS é vinculado ao PID do processo herdeiro no momento da criação da tabela, e nada no sistema realinha esse vínculo quando o herdeiro reinicia — a vulnerabilidade persiste por todo o intervalo entre o *crash* do herdeiro e o próximo *crash* do dono, que pode ser arbitrariamente longo.

A correção óbvia, um segundo herdeiro que protege o primeiro, não fecha essa janela: ela apenas a desloca um nível acima, sujeita à mesma coincidência de timing entre o segundo herdeiro e o primeiro. A correção estrutural é não ter herdeiro: AFL.BufferKeeper e AFL.ModelKeeper

foram reescritos para possuir suas tabelas ETS *permanentemente* (posse invertida), nunca as repassando ao worker via `:ets.give_away/3`. Como nenhum dos dois tem lógica de domínio (nenhuma agregação, nenhum tratamento de gradiente), sua própria superfície de crash é próxima de zero, e o crash do *worker* (EdgeNode ou Aggregator) deixa de poder tocar a tabela por construção, e não só na prática. Para o `AFL.ModelKeeper` especificamente, onde a perda dói mais (o modelo global inteiro), a correção escala além do processo/ETS: cada atualização também grava um checkpoint atômico em disco (arquivo temporário + `File.rename!/2`, atômico em POSIX), relido no próprio `init/1` caso este processo, ou a BEAM inteira, reinicie.

A Figura 3(b) mostra o resultado desse redesenho sob o teste que realmente importa depois da correção: não mais “herdeiro morre, depois o dono”, mas diretamente “o dono morre”, já que não há mais herdeiro para matar primeiro. Medido sobre 20 *trials* por cenário: matar `AFL.ModelKeeper` diretamente resulta em **0 das 100 rodadas perdidas (0%)**. O checkpoint em disco fecha o gap por completo para o modelo global, o caso que mais importava. Matar `AFL.BufferKeeper` diretamente, por outro lado, resulta em **60 dos 60 gradientes perdidos (100%)**: sem persistência em disco para o buffer, a tabela morre com seu dono, e a árvore `:rest_for_one` apenas garante que o EdgeNode reinicia em cascata de forma consistente, não que o conteúdo sobreviva. Este é um resíduo **aceito e caracterizado**, não uma omissão: a redundância no nível de processo não se fecha por composição (proteger um processo com outro apenas move a vulnerabilidade para o segundo); eliminá-la por completo exige a mesma escalada para uma camada de persistência aplicada ao modelo, deixada como trabalho futuro imediato (Seção VIII), já que a superfície de crash de um processo com zero lógica de domínio, embora pequena, não é literalmente zero (finalização pelo supervisor, `Process.exit/2` externo, queda da BEAM).

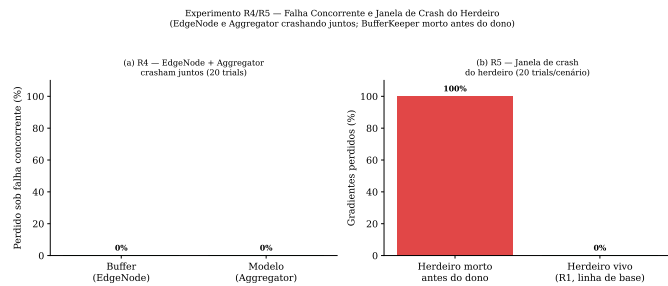


Figura 3: Experimento R4/R5. (a) Perda de gradientes e de rodadas de treino sob falha concorrente (EdgeNode e Aggregator crashando juntos), 20 *trials*. (b) Perda sob crash direto do dono — `AFL.BufferKeeper` (buffer) ou `AFL.ModelKeeper` (modelo) — já sob o redesenho de posse invertida com checkpoint em disco para o modelo.

F. Validação em Rede Real: a Simulação se Sustenta

A Figura 4 mostra o protocolo da Seção V-D: o mesmo cenário de *stragglers* do Experimento A, mas entre nós BEAM

genuinamente distribuídos em containers Docker conectados por TCP/IP real. Sobre 10 repetições: throughput assíncrono de 144.4 ± 1.3 upd/s (contra 153.3 ± 0.1 upd/s simulado, uma diferença de apenas 5,8%, atribuível ao overhead genuíno de rede/serialização ausente da simulação de VM única); throughput síncrono de 9.9 ± 0.1 upd/s (essencialmente idêntico ao simulado, 9.9 ± 0.0 upd/s, como esperado, já que o síncrono é dominado pelo *straggler* de 500 ms, real ou simulado); e speedup de $14.62 \times \pm 0.19$, mesma ordem de grandeza do $15.44 \times$ simulado. A conclusão central do Experimento A, de que a barreira síncrona, não o conteúdo computacional do update, é o gargalo, não é um artefato da simulação de latência via `Process.sleep`: sustenta-se sob rede TCP/IP genuína, com uma pequena e explicável penalidade de overhead real no braço assíncrono, e nenhuma penalidade mensurável no síncrono (que já era, por construção, limitado pelo nó mais lento). Uma ressalva honesta: a rede aqui é a bridge interna do Docker, com latência de trânsito sub-milissegundo entre containers no mesmo hospedeiro, não uma rede geograficamente distribuída como um deploy real de Edge Computing veria; o objetivo deste experimento é validar que a pilha de rede real (TCP, serialização, *scheduling* do sistema operacional) não muda a conclusão qualitativa, não reproduzir latência de WAN.

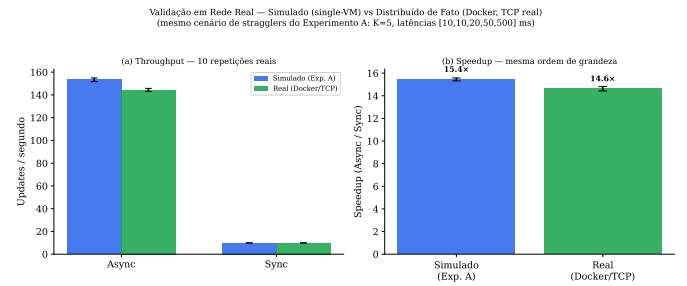


Figura 4: Validação em rede real. (a) Throughput assíncrono e síncrono, simulado (Experimento A, VM única) vs distribuído de fato (Docker, TCP real), 10 repetições reais. (b) Speedup resultante — mesma ordem de grandeza em ambos os regimes.

G. Overhead Real de Memória do Ring Buffer

O Experimento E mede o footprint exato de $B_{\max} = 5$ entradas do ring buffer: 40 bytes/entrada (200 bytes no total) para o tensor sintético $d = 10$, contra 101.800 bytes/entrada (≈ 497 KB no total) para o MLP real $784 \rightarrow 32 \rightarrow 10$: um fator $2.545 \times$ maior. Mesmo esta rede pequena já consome quase 0,5MB por nó apenas no ring buffer; a Seção VII discute a extrapolação para modelos de produção.

VII. DISCUSSÃO

A. O que o Gap de Resiliência Revela

O resultado mais importante deste trabalho (Seções VI-A e VI-B) não é sobre FL especificamente. É sobre a diferença entre *usar* uma plataforma com tolerância a falhas e *verificar* que essa tolerância se aplica a todos os componentes do próprio sistema. O AFL sempre citou “BEAM/OTP” e “árvore

de supervisão” como justificativa de resiliência (Seção III), e essa citação não era falsa: o Aggregator de fato era, desde a primeira versão, supervisionado e recuperado corretamente. O que a citação omitia, e que só a injeção de falhas sistemática expôs, é que *nem todo processo do sistema estava sob essa mesma árvore*. Isso é uma classe de erro estruturalmente diferente de um bug de lógica de negócio: o código do EdgeNode em si estava correto (a lógica de *ring buffer*, a máquina de estados, a matemática de *flush*). O gap estava inteiramente na *composição* do sistema, algo que revisão de código dificilmente detecta e que só se manifesta sob a condição específica de um *crash* do próprio processo (não do seu vizinho). Isso sugere que qualquer reivindicação de resiliência baseada na plataforma escolhida, não só em FL, não só em BEAM, deveria vir acompanhada de evidência de injeção de falhas cobrindo cada processo do sistema individualmente, e não só os mais críticos ou mais fáceis de testar.

B. Limitações

- 1) **Escala do modelo:** os Experimentos B/D/E usam um MLP de 25.450 parâmetros. O microbenchmark de escala (Seção VII, item de overhead de checkpoint) já ancora essa preocupação num ponto real, e não só especulativo: para o MLP de 1.333.770 parâmetros testado ali (52,4× maior), um ring buffer com $B_{\max} = 5$ ocuparia $\approx 26,7$ MB por nó — medido diretamente, não extrapolado. Modelos de produção têm 10^6 – 10^{10} parâmetros, um intervalo que ainda inclui escalas muito além da testada aqui; extrapolando linearmente a partir do byte count real medido, um modelo com 10^7 parâmetros ocuparia ≈ 190 MB só no ring buffer de um único nó. A ETS torna-se gargalo real de memória nessa escala, e não só hipotético, ainda que a validação direta permaneça limitada a duas ordens de grandeza acima da arquitetura original, não às 10 ordens do limite superior de produção.
- 2) **Privacidade:** O AFL nesta forma não aplica Privacidade Diferencial [10]. A integração com DP é deixada como trabalho futuro.
- 3) **Escopo do Experimento R, parcialmente estendido:** o Experimento R4 (Seção VI-D) já testa EdgeNode e Aggregator crashando juntos, sem interação problemática encontrada. Mas AFL.EdgeNode ainda usa um nome registrado fixo (`{:local, __MODULE__}`), permitindo apenas uma instância supervisionada por vez — não se testou *múltiplos* EdgeNodes falhando em cascata, nem particionamento de rede (que difere de *crash* de processo: o processo continua vivo, apenas inatingível). Generalizar AFL.EdgeNodeSupervisor para múltiplas instâncias nomeadas dinamicamente e caracterizar falhas correlacionadas entre vários EdgeNodes é extensão natural deste experimento.
- 4) **Residual de resiliência — corrigido para o modelo, aceito e caracterizado para o buffer:** o Experimento R5 (Seção VI-E) mostra que a janela de crash do herdeiro (`:heir` de uma tabela ETS vinculado ao PID do herdeiro no momento da criação da tabela, nunca rea-

linhado se o herdeiro reinicia) é um gap real e total, não uma corrida estreita. A correção — posse invertida (AFL.BufferKeeper/ AFL.ModelKeeper possuem suas tabelas permanentemente) mais checkpoint em disco para o AFL.ModelKeeper — fecha o gap por completo para o modelo global (0% de perda ao matar o dono, medido). Para o buffer de gradientes, sem persistência em disco, a perda ao matar o dono permanece em 100% — um resíduo aceito, não uma corrida silenciosa: redundância no nível de processo não se fecha por composição (proteger um processo com outro só desloca a vulnerabilidade), e fechar esse resíduo por completo exigiria a mesma escalada para disco já aplicada ao modelo (trabalho futuro, Seção VIII).

- 5) **Overhead de checkpoint mensurável, e sub-linear na escala testada, ao contrário do que se assumia:** medido diretamente (microbenchmark isolado, 1.000 chamadas a `AFL.Aggregator.update_sync/2` com e sem a linha de `AFL.ModelKeeper.checkpoint/1`, 5 repetições de cada): $589,8 \mu\text{s}/\text{update}$ com *checkpoint* vs. $559,0 \mu\text{s}/\text{update}$ sem: overhead de $\approx 30,8 \mu\text{s}/\text{update}$ ($\approx 5,5\%$ relativo) para o MLP de 25.450 parâmetros usado neste trabalho. Uma versão anterior deste texto assumia, sem medir, que “o custo de um `:ets.insert` tende a crescer com o tamanho do estado serializado”. Testado diretamente aqui: isolando o custo do próprio `:ets.insert` do snapshot completo do estado (sem o custo do *merge* FedAvg em si) para um MLP 52,4× maior (1.333.770 parâmetros, `AFL.Model.build_large/0`, $784 \rightarrow 1024 \rightarrow 512 \rightarrow 10$), o tempo por inserção cresce de $1,08 \mu\text{s}$ para $1,50 \mu\text{s}$: um fator de apenas 1,4×, não os 52,4× que uma extrapolação linear preveria. Isso é consistente com o comportamento de *refc binaries* do Erlang: tensores grandes são referenciados, não copiados escalar-a-escalar, no processo de inserção na ETS, então o custo é dominado por overhead fixo de construção do termo, não pelo número de parâmetros. Isso não elimina a preocupação de memória do item 1 (o ring buffer ainda precisa *guardar* os bytes, e isso escala linearmente), mas refuta especificamente a hipótese de que o *tempo de checkpoint* seria o gargalo dominante em modelos maiores.

VIII. CONCLUSÃO

Este trabalho partiu de uma suposição comum em sistemas de FL construídos sobre plataformas com tolerância a falhas nativa, a de que a resiliência é uma propriedade herdada automaticamente da plataforma, e a submeteu a verificação empírica sistemática, via Chaos Engineering [5], em vez de aceitá-la por inspeção de código. O resultado central é que essa suposição estava **parcialmente falsa**, em dois componentes distintos do sistema. O processo EdgeNode não estava em nenhuma árvore de supervisão, e seu *ring buffer* ETS não sobrevivia ao próprio *crash* do processo que o criava: sob 60 gradientes injetados em 20 *crashes*, **100% eram perdidos** (R1, Seção VI-A). Mais grave, o Aggregator *era* corretamente supervisionado, mas seu estado não. Sob 100 rodadas de

treino injetadas em 20 *crashes*, **100% do progresso de treinamento era destruído** a cada *crash*, reiniciando sempre a partir de pesos estáticos, como se nenhum treinamento tivesse ocorrido (R2, Seção VI-B): um único ponto de falha capaz de apagar o modelo global inteiro. As correções (AFL.EdgeNodeSupervisor + AFL.BufferKeeper para o EdgeNode; AFL.ModelKeeper para o Aggregator, Seção III-G) eliminam ambas as perdas por completo sob o mesmo protocolo (0/60 e 0/100, respectivamente), com tempo de recuperação sub-milissegundo preservado (R3, Seção VI-C: mediana 0,23 ms para o Aggregator, 0,22 ms para o EdgeNode), mais de três ordens de grandeza mais rápido que orquestradores de contêiner convencionais, por não depender de re-provisionamento de sistema operacional. Esta é a contribuição central deste trabalho: não uma nova fórmula de agregação, mas uma metodologia de verificação de resiliência via injeção de falhas sistemática, e os gaps reais que ela expôs e permitiu corrigir com evidência antes/depois, em vez de argumento de design.

A extensão do escopo de falhas confirma que essas correções se sustentam sob estresse adicional, e revela um terceiro gap que expôs uma limitação estrutural do próprio padrão de correção usado nos dois primeiros. Sob falha concorrente (R4, Seção VI-D), com EdgeNode e Aggregator crashando juntos, não há interação problemática: a perda permanece em 0%, como nos casos isolados. Já o Experimento R5 (Seção VI-E) mostrou que o padrão de herdeiro ETS usado nas duas primeiras correções tem uma janela de corrida real e total (não um instante raro): se o próprio processo herdeiro morre antes do processo que ele protege, a perda é de 100%, porque o `:heir` de uma tabela ETS é vinculado a um *pid* no momento da criação da tabela, não re-vinculado quando o herdeiro reinicia; encadear um segundo herdeiro não fecharia essa janela, apenas a deslocaria um nível acima. A correção estrutural, posse invertida (o dono nunca repassa a tabela) combinada, para o modelo global, com checkpoint atômico em disco, fecha esse gap por completo onde ele mais importava (0% de perda ao matar o dono do modelo, contra 100% antes da correção), e deixa um resíduo aceito e caracterizado no buffer de gradientes (100%, sem persistência em disco), reportado como tal: um trade-off explícito entre o custo da correção completa e a criticidade de cada estado, não uma lacuna omitida. A validação em rede real (Seção VI-F, containers Docker com TCP genuíno em vez de `Process.sleep`) confirma que os números obtidos sob latência simulada numa única VM BEAM não escondiam nenhum efeito de rede real relevante para essas conclusões.

A implementação em Elixir/BEAM demonstra que a plataforma OTP oferece primitivos nativos (`gen_statem`, `Process.monitor`, `:ets`) que mapeiam diretamente sobre os requisitos de sistemas de FL resilientes, reduzindo a complexidade de implementação de tolerância a falhas *quando corretamente fiados a cada processo*: os três gaps descobertos neste trabalho (Seções III-G, VI-A–VI-B, VI-E) mostram que esses primitivos não protegem automaticamente um processo deixado fora da árvore de supervisão, nem por composição

ingênua de mais processos do mesmo tipo (herdeiro de herdeiro). A plataforma reduz o esforço de implementação da resiliência, mas fechar um gap de verdade, às vezes, exige trocar de camada de abstração (processo/ETS para persistência em disco), em vez de simplesmente adicionar mais um processo; e essa diferença só é visível por verificação empírica, não por inspeção de código.

Como trabalhos futuros, propõe-se, em ordem de prioridade: (i) aplicar a mesma escalada para disco usada no AFL.ModelKeeper ao AFL.BufferKeeper, fechando o resíduo aceito do Experimento R5 (Seção VI-E) para o buffer de gradientes também, provavelmente via um formato *append-only* (WAL) em vez de um snapshot completo por escrita, já que o buffer muda a cada gradiente, não a cada rodada; (ii) generalizar AFL.EdgeNodeSupervisor para múltiplas instâncias nomeadas dinamicamente e caracterizar falhas em cascata entre vários EdgeNodes: o Experimento R4 (Seção VI-D) já descarta interação problemática entre EdgeNode e Aggregator crashando juntos, mas não cobre múltiplos EdgeNodes simultâneos; (iii) validação com modelos de escala de produção real (CNNs/Transformers, $> 10^8$ parâmetros): o microbenchmark de checkpoint (Seção VII) já cobre 1.333.770 parâmetros (52,4× acima do MLP original), mas isso ainda é 3-4 ordens de grandeza abaixo do limite superior de produção (10^{10}), e usa um MLP maior, não uma arquitetura de produção real (CNN/Transformer) — para testar se a escala sub-linear observada no overhead de checkpoint (Seção VII) se sustenta em arquiteturas e escalas muito maiores, ou se é específica da faixa testada; (iv) repetir a Validação em Rede Real (Seção V-D) sobre uma rede geograficamente distribuída de fato (múltiplas máquinas físicas ou regiões de nuvem), diferente da bridge local do Docker usada nesta versão; (v) caracterizar como a capacidade do ring buffer ($B_{\max}$) e uma política adaptativa de dimensionamento afetam a tolerância a desconexões prolongadas sob o mesmo protocolo de injeção de falhas do Experimento R1.

REFERÊNCIAS

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 54, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [3] Q. Ho, J. Cipar, H. Cui, S. Lee, J. K. Kim, P. B. Gibbons, G. A. Gibson, G. Ganger, and E. P. Xing, “More effective distributed ML via a stale synchronous parallel parameter server,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 26, 2013, pp. 1223–1231.
- [4] C. Xie, S. Koyejo, and I. Gupta, “Asynchronous federated optimization,” in *Workshop on Optimization for Machine Learning, NeurIPS 2019*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.03934>
- [5] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [6] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander, “Towards federated learning at scale: A system design,” in *Proceedings of the 2nd SysML Conference*, Palo Alto, CA, USA, 2019. [Online]. Available: <https://arxiv.org/abs/1902.01046>

- [7] Y. Shi, K. Yang, T. Jiang, J. Zhang, and K. B. Letaief, "Communication-efficient edge AI: Algorithms and systems," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2167–2191, 2020.
- [8] Elixir Core Team, "Elixir programming language," 2024, version 1.19. [Online]. Available: <https://elixir-lang.org>
- [9] Nx Contributors, "Nx: Numerical elixir," 2024, version 0.11. [Online]. Available: <https://github.com/elixir-nx/nx>
- [10] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.